Manual Book

Watcher & Callback

- CLI Script
  - Cara Pakai
  - Cara upload CLI Script
  - Cara Membuat Kode untuk CLI Script
    - Struktur Kode
  - Fungsi Custom yang sudah ada di CLI Script
  - Cara Membuat Unit Test untuk CLI Script
  - Penggunaan Library
- Watcher
  - Cara Pakai
    - Buat CLI script yang akan dijalankan.
    - Buat Tabel yang diperlukan pada database Biz-A
    - Set Watcher yang diinginkan
    - Jalankan Watcher Server (Biz-A CLI)
  - Watcher History
- Callback
  - Cara Pakai
    - Buat Script
    - Jalankan hub
    - Buat Callback Link
    - Test Request bisa dicoba dari Postman atau Thunder client. Contohnya:

CLI Script

Cara Pakai

- Gunakan Node minimal **20.12.0**
- Gunakan biz-a-cli minimal **2.3.64**

- Jika Watcher atau Callback muncul error tidak dapat menemukan link import ketika menjalankan watcher atau callback pada cmd, maka solusinya adalah pindah folder referensi, baru jalankan watcher atau callback. Hal ini disebabkan file cli script meng-import module dengan referensi folder biz-a-cli.
  Solusi yang bisa dilakukan adalah dengan mengganti folder dengan format:
  cd "C:\Users\<nama user>\AppData\Roaming\npm\node_modules\biz-a-cli"

  Contoh :
  cd "C:\Users\Operational\AppData\Roaming\npm\node_modules\biz-a-cli"

Tabel CLI_SCRIPT akan terbentuk secara otomatis pada database Biz-A dengan statement:

```
SET TERM ^ ;

CREATE GENERATOR SYS$CLI_SCRIPT_ID_GEN^
CREATE TABLE SYS$CLI_SCRIPT
(
  ID INTEGER NOT NULL,
  SCRIPT_NAME VARCHAR(256) NOT NULL,
  SCRIPT BLOB SUB_TYPE 0 NOT NULL,
  CONSTRAINT PK_SYS$CLI_SCRIPT_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$CLI_SCRIPT_SCRIPT_NAME UNIQUE (SCRIPT_NAME)
)^

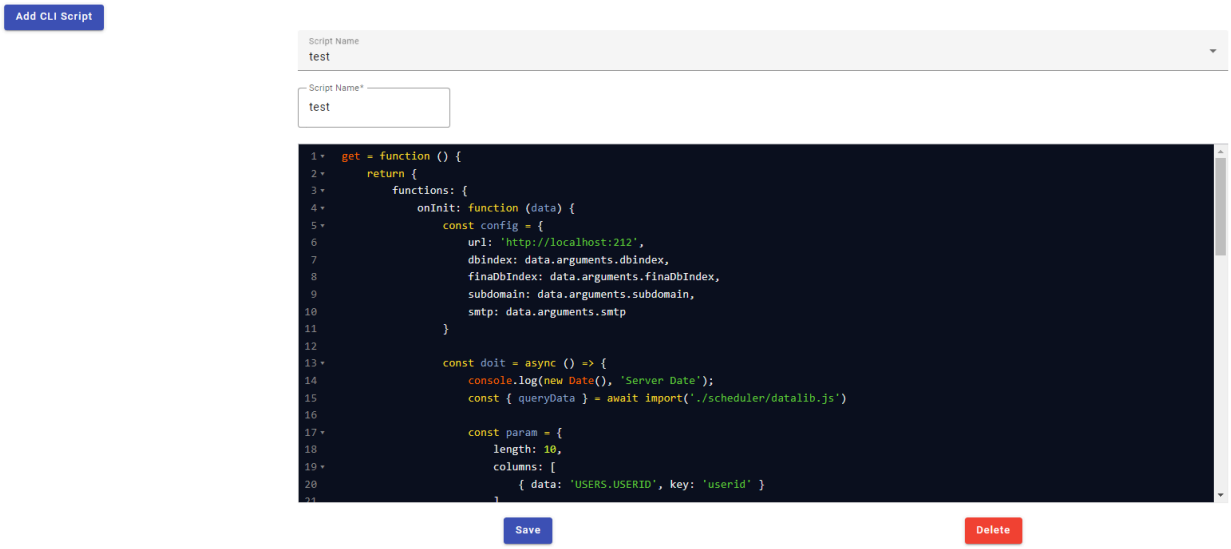

CREATE TRIGGER SYS$CLI_SCRIPT_BI FOR SYS$CLI_SCRIPT ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$CLI_SCRIPT_ID_GEN, 1);
END^

SET TERM ; ^
```

Cara upload CLI Script

-Buat script yang akan dijalankan pada table CLI_SCRIPT dengan cara **masuk ke menu Admin dan cari SubMenu CLI Script.**
-Jika mau membuat CLI Script yang baru, klik Add CLI Script agar bersih dahulu jika ada input yang terisi.
-Jika mau mengubah CLI Script, maka pilih dahulu script yang akan diubah, kemudian ubah dan simpan.

-Jika mau menghapus CLI Script, maka pilih script kemudian hapus.
-Pada bagian Script Name, tidak perlu menggunakan extension .js. Cukup nama file saja.



Cara Membuat Kode untuk CLI Script

Struktur Kode

Struktur kode-nya harus seperti ini:

```
get = function (lib) {
    return {
        functions: {
            func1: function(){
            },
            onInit: function (data) {
                <isi dengan fungsi yang akan dijalankan>
            }
        }
    }
}
```

Keterangan:

1. lib adalah library yang sudah disediakan oleh CLI Script. Lib yang tersedia adalah:
    a. axios (**promised based**, bukan observable)
    b. dayjs
    c. crypto

    d. log
    e. data: library untuk FINA API. Cara pakainya silakan lihat manual book Biz-A.
        i. sendModel
        ii. queryData
        iii. crudData
        iv. genId
        v. getUrlAndParam

Untuk axios pada poin a, saat ini axios sudah ditambahkan timeout default 30s, Jika ingin diganti, bisa dengan menambahkan “options” dan set “timeout”-nya ketika axios dipanggil.

Untuk data di poin e, saat ini library tersebut sudah ditambahkan timeout default 30s. Jika mau timeout mau diganti secara keseluruhan, set pada apiConfig, tambahkan field "timeout". Jika diganti hanya untuk spesifik fungsi tertentu saja. apiConfig bisa di-clone dahulu dan set timeout.

2. Fungsi **onInit** wajib ada. Fungsi ini yang akan dijalankan oleh callback maupun watcher.
    a. Parameter data berisi:
        i. arguments: object. Berisi semua arguments saat menjalankan Biz-A CLI.
        ii. body: body dari callback request.
        iii. path: string, merupakan path tambahan di belakang /cb
        iv. query: object: merupakan isi semua query param dari request.
        v. headers: object: merupakan header dari request.
        vi. socket: socket instance yang menghubungkan Biz-A CLI ke Biz-A Server.
    b. Return adalah respon apa pun yang diharapkan oleh callback.
3. Selain onInit, apabila diperlukan, bisa dibuat fungsi-fungsi yang lain setingkat dengan onInit. Cara memanggil antar fungsi harus menggunakan **this**. Agar **this** bisa terbaca, pastikan pembuatan fungsi menggunakan **function**, bukan **=>** (arrow function) karena arrow function akan menyebabkan this-nya ke diri dia sendiri.
4. Untuk menggunakan library, gunakan sintaks **await import**. Contohnya dapat dilihat di sini.
5. Jika ada exception yang dibalikkan oleh axios, maka kita bisa mendapatkan informasi yang diperlukan dengan mengikuti struktur objectnya, yaitu :

```
export class AxiosError<T = unknown, D = any> extends Error {
  constructor(
    message?: string,
    code?: string,
    config?: InternalAxiosRequestConfig<D>,
    request?: any,
    response?: AxiosResponse<T, D>
  );
```

```
export interface AxiosResponse<T = any, D = any> {
  data: T;
  status: number;
  statusText: string;
  headers: RawAxiosResponseHeaders | AxiosResponseHeaders;
  config: InternalAxiosRequestConfig<D>;
  request?: any;
}
```

Sebagai contoh jika ada error dari Database, kita bisa menangkap error tersebut dari :

```
Try {
…
}
catch(error) {
   console.error(error.response.data);
}
```

Fungsi Custom yang sudah ada di CLI Script

- **queryData**, tambahkan const { queryData } = await import('./scheduler/datalib.js');
- **sendModel**, tambahkan const { sendModel } = await import('./scheduler/datalib.js');
- **json2fina**, tambahkan const { json2fina } = await import('./scheduler/datalib.js');
- ~~**sendMailWatcher,** tambahkan const { sendMailWatcher } = await import('./mailController.js');~~
- **sendMailCliScript,** tambahkan const { sendMailCliScript } = await import('./mailController.js');
    - Provider:
        - **IndiHome, XL** gunakan setting port: 465, secure: true
        - **Starlink** gunakan setting port: 587, secure: false

Note:

- Jika diperlukan module lain selain list di atas (selain default), bisa dicoba dengan:
    - Install module yang diperlukan, contoh: ““npm install winston“ atau “npm install oracledb“.
    - Import pada cliScript dengan contoh:
        - Untuk modul yang sudah ESM: const { winston } = await import('winston'')
        - Untuk modul yang masih CommonJs: const { default: oracledb } = await import('oracledb')

Cara Membuat Unit Test untuk CLI Script

Berikut langkah-langkah agar bisa membuat unit test untuk CLI Script:

1. Asumsinya code CLI Script di taruh di sub folder template \cli-script (nama sub folder bisa apa saja).
2. Masuk ke CMD ke folder template
3. Install jest dengan cara ketik: `npm i jest --save-dev`.
4. Buka file package.json, lalu tambahkan baris berikut di bawah scripts:

```
"jest": "node --experimental-vm-modules node_modules/jest/bin/jest.js
--watch a"
```

5. Masih di cmd, jalankan npm run jest. Jest akan jalan dan mencari semua file dengan ekstensi test.js.
6. Buat file baru di subfolder cli-script dengan ekstensi test.js. Isi file tersebut kurang lebih sebagai berikut:

```
1  const axios = require("axios");
2  const crypto = require("crypto");
3  const dayjs = require("dayjs");
4  const lib = {
5    axios:null,
6    crypto:crypto,
7    dayjs:dayjs
8  }
9  const f = require('./file_cli')(lib).functions;
10 test('test fungsi abc', ()=>{
11   expect(f.abc()).toBeDefined();
12 })
```

Penjelasan:

1. Baris 1 s/d 8 adalah untuk membentuk parameter lib. Bila tidak membutuhkan lib, isi lib dengan null.
2. Baris 9 berfungsi untuk load file cli yang akan di-test. Dalam contoh ini nama file-nya adalah `file_cli.js` dan diletakan di folder yang sama dengan file test.js.
3. Baris 10 & 12 adalah pembuka & penutup sebuah unit test.
4. Baris 11 adalah contoh cara memanggil fungsi dan mengetes hasil-nya. Dalam contoh ini, nama fungsi yang dipanggil adalah abc.

Penggunaan Library

Saat ini CLI Script bisa menggunakan Library sehingga kode bisa lebih DRY. Caranya:

- Buat lib yang diperlukan. Cara pembuatannya sama dengan pembuatan CLI Script. Bedanya, onInit tidak wajib ada karena tidak dipanggil.
- Set pada Callback library yang diperlukan dengan cara :
  useLibrary: function () {

return [<nama library yang akan dipanggil>];
},

- Panggil library tersebut dengan: lib.<nama file library yang terset di useLibrary>.<nama fungsi atau konstanta yang dipanggil>

Contoh: libCb.js

```
get = function (lib) {
    return {
        functions: {
            TIME_FORMAT_PG: 'YYYY-MM-DDTHH:mm:ssZ',
            TIME_FORMAT_FB: 'YYYY-MM-DD HH:mm:ss',
            CURRENCY: 'IDR',
            httpMethod: 'POST',
            k: function () {
                return {
                    SUCCESS: 0,
                    DUPLICATE_PMT_REQ_ID: 1,
                    BILL_NOT_FOUND: 2,
                    INVALID_AMOUNT: 3,
                    TRANSACTION_NOT_FOUND: 4,
                }
            },
            expireTime: function (time) {
                return lib.dayjs(time, this.TIME_FORMAT_PG)
                    .add(24, 'hour')
                    .format(this.TIME_FORMAT_PG)
            },
            apiConfig: function (arg) {
                return {
                    dbindex: arg.dbindex,
                    url: `http://${arg.hostname}:${arg.port}`,
                }
            },
        }
    }
}
```

Contoh: createPayment.js

```
get = function (lib) {
    return {
```

```
        functions: {
            useLibrary: function () {
                return ['libCb'];
            },
            doInitialization: async function (data) {
                const { queryData } = await import('./scheduler/datalib.js')

                let param = {
                    filter: [{ junction: '', column: 'OPTIONS.OPTIONKEY',
operator: '=', value1: `'ESPAY'` }],
                    columns: [{ data: 'OPTIONS.OPTIONVALUE', key: 'value' }]
                }
                let res = await queryData(param,
lib.libCb.apiConfig(data.arguments), true, true)
                if (res.length == 0) {
                    return {
                        responseCode: "5004700",
                        responseMessage: 'Please initialization OPTION.ESPAY.'
                    }
                }
                globalThis.espay = JSON.parse(res[0].value)
                return '';
            },
            onInit: async function (data) {//harus pakai function, bukan => biar
bisa baca "this"
                const axios = lib.axios;
                const { sendModel } = await import('./scheduler/datalib.js');
                const { format } = await import('winston')

                lib.log.format = format.combine(
                    format.timestamp(),
                    format.json()
                )

                const init = await this.doInitialization(data)
                if (init.responseCode) {
                    return init;
                }
                return 'OK';
```

```
            }
        }
    }
}
```

Watcher

Management by Exception (Watcher) digunakan untuk mengontrol sesuatu hal sesuai dengan jadwal yang telah diset sebagai pengganti manusia dalam hal pengawasan. Jika ada hal yang berjalan di luar yang seharusnya atau disepakati maka akan muncul Notifikasi.

Contoh kasus:

1. Apabila sampai jam 9:10 karyawan belum absen, regional manajer akan dinotifikasi oleh Biz-A sehingga manajer yang bersangkutan bisa mengambil tindakan dengan cara, misalnya, menelepon yang bersangkutan.
2. Untuk salesman distributor yang mengunjungi outlet:
    a. bila dalam waktu yang sudah ditetapkan belum tiba di lokasi yang sudah ditentukan
    b. bila order yang baru masuk tidak memenuhi spread atau value tertentu
    c. Bila keluar dari jalur yang sudah ditetapkan
    d. Diam di 1 titik (warteg/outlet) lebih dari waktu tertentu

    Manajer yang bersangkutan bisa mendapat notifikasi sehingga bisa diambil langkah selanjutnya selagi salesmannya masih di lapangan.
3. Uang Muka Dinas yang sudah diambil belum dibuat pertanggungjawaban dalam waktu tertentu
4. Piutang customer yang belum dibayar setelah sekian hari dari jatuh tempo, atau sekedar pengingat sekian hari sebelum jatuh tempo akan dinotifikasi via email.

Cara Pakai

Buat CLI script yang akan dijalankan.

Lihat caranya di sini.

Contoh CLI script watcher untuk mengecek karyawan yang sudah terdata:

```
get = function () {
    return {
```

```
        functions: {
            onInit: function (data) {
                const config = {
                    url: 'http://localhost:212',
                    dbindex: data.arguments.dbindex,
                    finaDbIndex: data.arguments.finaDbIndex,
                    subdomain: data.arguments.subdomain,
                    smtp: data.arguments.smtp
                }

                const doit = async () => {
                    console.log(new Date(), 'Server Date');
                    const { queryData } = await
import('./scheduler/datalib.js')

                    const param = {
                        length: 10,
                        columns: [
                            { data: 'USERS.USERID', key: 'userid' }
                        ],
                        dbIndex: 1
                    }

                    const res = await queryData(param, config, true);
                    console.log(res, 'subscribe')
                }

                const sendEmail = async () => {
                    const { sendMailCliScript } = await
import('./mailController.js')

                    const mail = {
                        manager: 'SCY',
                        emailManager: 'samuel.cahya@imamatek.com',
                        nama: 'YCS'
                    }

                    const req = {
                        body: {
```

```
                            config: config,
                            mailOptions: {
                                from: 'Imamatek
<samuel.cahya@imamatek.com>',
                                to: `${mail.manager}
<${mail.emailManager}>`,
                                subject: 'Info: Pengumuman',
                                text: `Halo ${mail.manager},
              Staf anda ${mail.nama} terdata.
              Silahkan cek di Biz-A.

              Terima kasih.
              `,
                                html: `<p>Halo ${mail.manager},</P><br>
              <p>karyawan anda ${mail.nama} terdata.</p><br>
              <p>Silahkan dilihat di <a
href="https://biz-a.id/">BIZ-A</a>.</p><br>
              <br>
              <p>Terima kasih.</P>
              `,
                            },
                        }
                    }
                    await sendMailCliScript(req)
                }

                doit();
                sendEmail();

                return 'OK'
            }
        }
    }
}
```

Buat Tabel yang diperlukan pada database Biz-A

Tabel di bawah ini akan terbentuk secara otomatis dengan statement sebagai berikut:

```
SET TERM ^ ;

CREATE GENERATOR SYS$WATCHER_ID_GEN^
CREATE TABLE SYS$WATCHER
(
  ID INTEGER NOT NULL,
  COMPANY_OBJ_ID VARCHAR(24) NOT NULL,
  CONSTRAINT PK_SYS$WATCHER_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$WATCHER_COMPANY_ID UNIQUE (COMPANY_OBJ_ID)
)^

CREATE TRIGGER SYS$WATCHER_BI FOR SYS$WATCHER ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$WATCHER_ID_GEN, 1);
END^

SET TERM ; ^


SET TERM ^ ;

CREATE GENERATOR SYS$TIMER_ID_GEN^
CREATE TABLE SYS$TIMER
(
  ID INTEGER NOT NULL,
  WATCHER_ID INTEGER NOT NULL,
  ACTIVE SMALLINT DEFAULT 0 NOT NULL,
  NAME VARCHAR(40) NOT NULL,
  SEQ INTEGER NOT NULL,
  TIME_ZONE VARCHAR(40) NOT NULL,
  SCRIPT_ID INTEGER NOT NULL,

  CONSTRAINT PK_SYS$TIMER_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$TIMER_ID_SEQ UNIQUE (ID, SEQ)
)^

ALTER TABLE SYS$TIMER ADD CONSTRAINT FK_SYS$TIMER_1
  FOREIGN KEY (WATCHER_ID) REFERENCES SYS$WATCHER (ID)^
ALTER TABLE SYS$TIMER ADD CONSTRAINT FK_SYS$TIMER_2
  FOREIGN KEY (SCRIPT_ID) REFERENCES SYS$CLI_SCRIPT (ID)^
```

```
CREATE TRIGGER SYS$TIMER_BI FOR SYS$TIMER ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$TIMER_ID_GEN, 1);
END^

SET TERM ; ^


SET TERM ^ ;

CREATE GENERATOR SYS$DAILY_ID_GEN^
CREATE TABLE SYS$DAILY
(
  ID INTEGER NOT NULL,
  TIMER_ID INTEGER NOT NULL,
  DAYS VARCHAR(20),
  CONSTRAINT PK_SYS$DAILY_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$DAILY_TIMER_ID UNIQUE (TIMER_ID)
)^

ALTER TABLE SYS$DAILY ADD CONSTRAINT FK_SYS$DAILY_1
  FOREIGN KEY (TIMER_ID) REFERENCES SYS$TIMER (ID)^

CREATE TRIGGER SYS$DAILY_BI FOR SYS$DAILY ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$DAILY_ID_GEN, 1);
END^

SET TERM ; ^


SET TERM ^ ;

CREATE GENERATOR SYS$WEEKLY_ID_GEN^
CREATE TABLE SYS$WEEKLY
(
  ID INTEGER NOT NULL,
  TIMER_ID INTEGER NOT NULL,
  ORDINAL VARCHAR(20),
  DAYS VARCHAR(20),
```

```
  CONSTRAINT PK_SYS$WEEKLY_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$WEEKLY_TIMER_ID UNIQUE (TIMER_ID)
)^

ALTER TABLE SYS$WEEKLY ADD CONSTRAINT FK_SYS$WEEKLY_1
  FOREIGN KEY (TIMER_ID) REFERENCES SYS$TIMER (ID)^

CREATE TRIGGER SYS$WEEKLY_BI FOR SYS$WEEKLY ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$WEEKLY_ID_GEN, 1);
END^

SET TERM ; ^


SET TERM ^ ;

CREATE GENERATOR SYS$MONTHLY_ID_GEN^
CREATE TABLE SYS$MONTHLY
(
  ID INTEGER NOT NULL,
  TIMER_ID INTEGER NOT NULL,
  DAYS VARCHAR(100),
  CONSTRAINT PK_SYS$MONTHLY_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$MONTHLY_TIMER_ID UNIQUE (TIMER_ID)
)^

ALTER TABLE SYS$MONTHLY ADD CONSTRAINT FK_SYS$MONTHLY_1
  FOREIGN KEY (TIMER_ID) REFERENCES SYS$TIMER (ID)^

CREATE TRIGGER SYS$MONTHLY_BI FOR SYS$MONTHLY ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$MONTHLY_ID_GEN, 1);
END^

SET TERM ; ^


SET TERM ^ ;
```

```
CREATE GENERATOR SYS$MINUTELY_ID_GEN^
CREATE TABLE SYS$MINUTELY
(
  ID INTEGER NOT NULL,
  TIMER_ID INTEGER NOT NULL,
  EVERYMIN INTEGER,
  TIME_FROM VARCHAR(5),
  TIME_TO VARCHAR(5),
  CONSTRAINT PK_SYS$MINUTELY_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$MINUTELY_TIMER_ID UNIQUE (TIMER_ID)
)^

ALTER TABLE SYS$MINUTELY ADD CONSTRAINT FK_SYS$MINUTELY_1
  FOREIGN KEY (TIMER_ID) REFERENCES SYS$TIMER (ID)^

CREATE TRIGGER SYS$MINUTELY_BI FOR SYS$MINUTELY ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$MINUTELY_ID_GEN, 1);
END^

SET TERM ; ^


SET TERM ^ ;

CREATE GENERATOR SYS$HOURLY_ID_GEN^
CREATE TABLE SYS$HOURLY
(
  ID INTEGER NOT NULL,
  TIMER_ID INTEGER NOT NULL,
  HOURS VARCHAR(500),
  CONSTRAINT PK_SYS$HOURLY_ID PRIMARY KEY (ID),
  CONSTRAINT UQ_SYS$HOURLY_TIMER_ID UNIQUE (TIMER_ID)
)^

ALTER TABLE SYS$HOURLY ADD CONSTRAINT FK_SYS$HOURLY_1
  FOREIGN KEY (TIMER_ID) REFERENCES SYS$TIMER (ID)^

CREATE TRIGGER SYS$HOURLY_BI FOR SYS$HOURLY ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
```

```
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$HOURLY_ID_GEN, 1);
END^

SET TERM ; ^


SET TERM ^ ;

CREATE GENERATOR SYS$HISTORY_ID_GEN^
CREATE TABLE SYS$HISTORY
(
  ID INTEGER NOT NULL,
  TIMER_ID INTEGER NOT NULL,
  LATEST_RUN TIMESTAMP,
  CONSTRAINT PK_SYS$HISTORY_ID PRIMARY KEY (ID)
)^

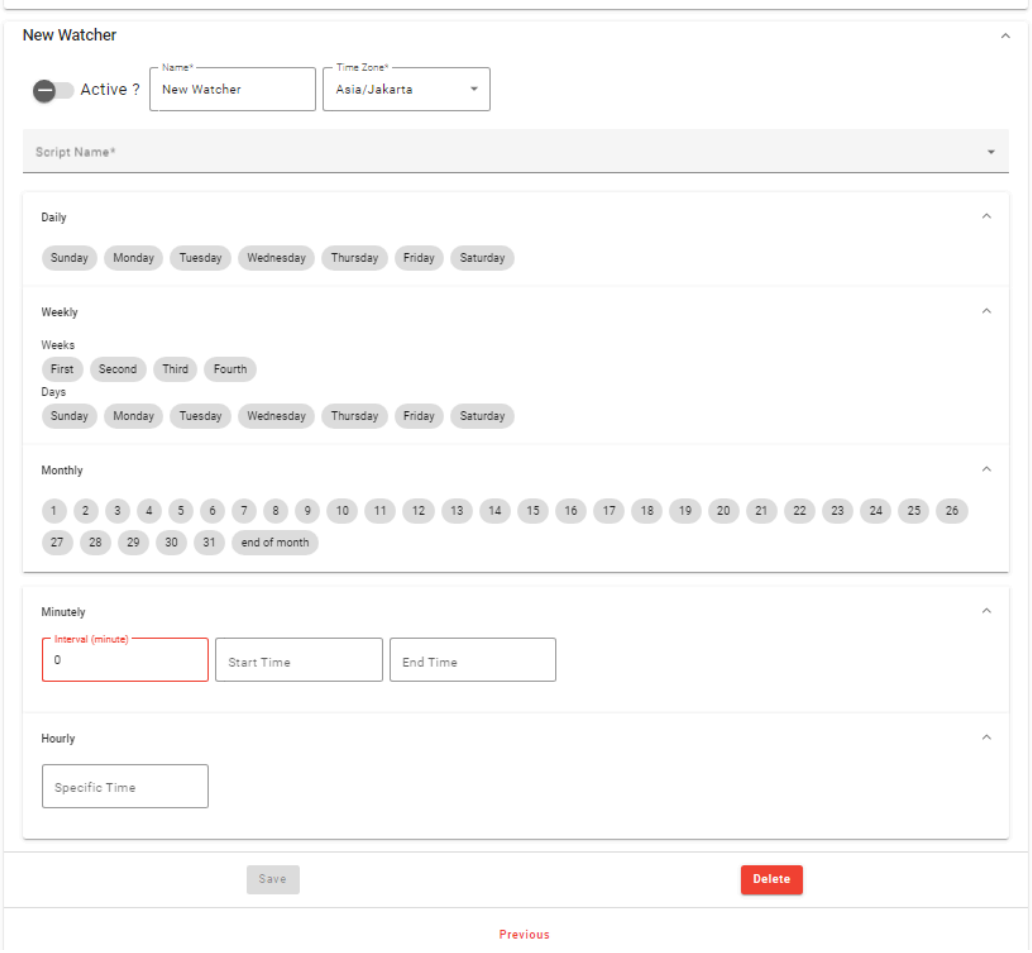
ALTER TABLE SYS$HISTORY ADD CONSTRAINT FK_SYS$HISTORY_1
  FOREIGN KEY (TIMER_ID) REFERENCES SYS$TIMER (ID)^

CREATE TRIGGER SYS$HISTORY_BI FOR SYS$HISTORY ACTIVE
BEFORE INSERT POSITION 0
AS
BEGIN
  IF (NEW.ID IS NULL) THEN new.ID = GEN_ID(SYS$HISTORY_ID_GEN, 1);
END^

SET TERM ; ^
```

Set Watcher yang diinginkan

- Masuk ke website biz-a, lihat bagian kanan atas ada menu **Admin**. Menu ini akan muncul jika user yang digunakan untuk login sudah dimasukkan ke dalam list Administrator oleh user Admin.
- Pilih **Watcher** dan pilih Add Watcher. Watcher bisa dibuat lebih dari 1.
- Cara membuat watcher sebagai berikut:



Keterangan:

- Active: Centang untuk mengaktifkan watcher.
- Name: Isi dengan nama watcher yang akan dibuat.
- TimeZone: Pilih Zona waktu yang akan dipakai dalam watcher. Untuk Indonesia ada 3 zona waktu yaitu **Asia/Jakarta**, **Asia/Makassar** dan **Asia/Jayapura**. Default = Asia/Jakarta.
- TemplateName: Isi dengan nama template script yang akan dijalankan. Contoh: cekUser atau libCekUser jika ingin menggunakan library.
- Bagian berikutnya dibagi menjadi 2, katakanlah bagian **Date** dan **Time**.
    - Bagian Date terdiri dari **Daily**, **Weekly** dan **Monthly** dan salah satu harus diisi.
      Daily: Pilih hari di mana Watcher akan jalan.
      Weekly:
        - ❖ Weeks: Pilih di minggu ke berapa Watcher akan dijalankan.
        - ❖ Days: Pilih hari di mana Watcher akan jalan.

      Monthly: Pilih di tanggal berapa Watcher akan dijalankan.

Ada pilihan juga apakah mau dijalankan di akhir hari bulan berjalan atau tidak.

- Bagian Time terdiri dari **Minutely** dan **Hourly** dan salah satu harus diisi.

  Minutely:
  - ❖ Interval: diisi dengan interval waktu yang diinginkan dalam satuan menit. Contoh, Watcher akan dijalankan 5 menit sekali, maka isi interval dengan 5.
  - ❖ StartTime: Waktu mulai (Interval akan dijalankan). Contoh, StartTime diset 13:00, maka watcher akan berjalan mulai jam 13:00 dan seterusnya. Watcher tidak berjalan sebelum jam 13:00.
  - ❖ EndTime: Waktu berakhir (Interval akan dihentikan). Contoh, EndTime diset 20:00, maka watcher akan berhenti mulai jam 20:00 dan seterusnya. Watcher tetap berjalan sebelum jam 20:00.

  Hourly:
  - ❖ Diisi dengan jam dan menit kapan watcher akan dijalankan. Bisa diisi lebih dari 1.

- Simpan Watcher.
- Jika tidak diperlukan lagi, bisa juga Delete Watcher.

Note: StartTime, EndTime dan Hourly saat ini diisi manual dahulu karena TimePicker yang dipakai tidak cocok. Akan diperbaiki.

Jalankan Watcher Server (Biz-A CLI)

Command: watcher
Options: -c <companyName>[required]
-p <port>[optional].

- Jalankan perintah “watcher -c <companyName>”..
- Ada juga opsi lain untuk mengganti port server (default port 3001) dengan “watcher -P <no port>”.
- Contoh pemakaian “watcher -c IMAMATEK -p 1234”.

Watcher sudah siap digunakan dan langsung berjalan.
Untuk test, bisa juga dilihat di dalam log apakah watcher sudah menjalankan scripting atau belum.

Jika menggunakan NVM, pastikan path ketika akan mengeksekusi watcher sudah benar. Umumnya letak path ada di C:\Users\DEV\AppData\Roaming\nvm\<versi node>\node_modules\biz-a-cli.

Watcher History

Saat ini, ada fitur untuk menjalankan scripting yang terlewat dikarenakan Biz-A CLI tidak berjalan.
Cara kerja fitur ini adalah dengan menyimpan waktu terakhir watcher berjalan di tabel SYS$HISTORY.

Dengan begitu, jika Biz-A CLI kembali aktif, maka akan mengambil data, kemudian akan menjalankan yang terlewat.

History akan disimpan untuk Watcher dengan status **Active.** Jika non aktif, maka tidak akan disimpan.
Jika Watcher dari aktif ke non aktif, maka history akan dihapus.

Callback

Requirement Callback dapat dilihat di sini.

Cara Pakai

Buat Script

Buat Script yang akan dijalankan seperti informasi pada Cara membuat CLI script.
Pastikan juga ada return pada fungsi onInit agar callback mendapatkan response.
Contoh file CLI script sebagai berikut::

```
get = function () {
    return {
        functions: {
            onInit: function (data) {
```

```
                const doit = async () => {
                    console.log(new Date(), 'Server Date');

                    const { queryData, sendModel } = await
import('./scheduler/datalib.js')
                    const param = {
                        length: 10,
                        columns: [
                            { data: 'USERS.USERID', key:
'userid' }
                        ],
                        dbIndex: 1
                    }

                    const config = {
                        url:
`http://${data.arguments.hostname}:${data.arguments.port}`,
                        dbindex: data.arguments.dbindex,
                        subdomain: data.arguments.subdomain
                    }

                const res = await queryData(param, config, true);
                    console.log(res, 'subscribe')
                }
                doit()
                return 'OK'
            }
        }
    }
}
```

Keterangan:
Fungsi **onInit** memiliki parameter **data** berisi:

```
    const data = {
            arguments: {
                    hostname: string,
```

```
                    port: number,
                    dbindex: number,
                    finaDbIndex: number,
                    subdomain: string,
                    smtp: {
                          auth: {
                              user: string,
                              pass: string
                          }
                          host: string,
                          port: number,
                          secure: boolean
                    }
    },
              body: object
         }
```

Arguments berasal dari parameter yang diinput saat command hub diaktifkan.
Body berasal dari data yang dikirim dari requester pada bagian body.

Jalankan hub

hub --server https://server.biz-a.id --sub scy --hostname localhost --port 212 --dbindex 2

Keterangan:
Untuk port dan dbindex adalah optional.
port digunakan untuk akses FINA API (default 212)
dbindex digunakan untuk akses set Index Biz-A Database (default 2)

Buat Callback Link

Lakukan test dengan mengambil link Callback yang bisa dibuat pada Biz-A.
Caranya adalah dengan masuk ke menu Admin, kemudian SubMenu Callback.



Test Request bisa dicoba dari Postman atau Thunder client. Contohnya:

Method : **POST**
https://server.biz-a.id/cb?subdomain=scy&companyid=SCY&scriptid=1

Body: (isi apapun yang diperlukan)

```
{
  "key1": "value1"
}
```

Keterangan:
https://server.biz-a.id/cb : link yang dipanggil untuk callback.
subdomain=scy : parameter subdomain untuk koneksi hub.
companyId=SCY : parameter company yang dipakai.
scriptid=1 : parameter id script yang akan dijalankan jika callback berhasil berkomunikasi.
